# 数学与 CS 问答笔记

math-answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 15, 2026

# Contents


1. 当日起点：项目状态与工具状态 ..... 3
   1.1. 进度概览 ..... 3
   1.2. 手上已有的工具（10-11 建的，今天直接可用） ..... 3
2. Go 有没有 assert？防御性编程在 Go 里怎么做 ..... 4
   2.1. 费曼直觉 ..... 4
   2.2. 硬核机制 ..... 4
      2.2.1. 官方态度（Go FAQ 原文） ..... 4
      2.2.2. 各语言断言的真实命运 ..... 4
      2.2.3. Go 的替代：三层防御 ..... 5
      2.2.4. 真要一个 assert，它长这样 ..... 5
   2.3. 落到你的项目：防御性编程具体怎么做 ..... 6
      2.3.1. 实测 1：`KV{}` 不 Open 就 Set，会怎样 ..... 6
      2.3.2. 实测 2：`Get` 未命中该不该报错 ..... 6
      2.3.3. 实测 3：跨 goroutine 的 panic 兜不住 ..... 7
      2.3.4. 决策表：这种情况该用哪个 ..... 7
   2.4. 陷阱与易错 ..... 7
   2.5. 关键点 ..... 8
   2.6. 变式检验 ..... 8
   2.7. 同类归类 ..... 8
   2.8. Go 语言机制补充 ..... 8
   2.9. 本次操作与当前状态 ..... 9
3. 把 defer 探针代码放进 scratch/defer/ ..... 10
   3.1. 这次做了什么 ..... 10
   3.2. 结果：输出与笔记逐字一致 ..... 10
   3.3. 本次操作与当前状态 ..... 11

Index of Listings ..... 12

# 1. 当日起点：项目状态与工具状态

这一节记录 10-13 开工时的状态（距上次记录 10-11 已过 2 天），作为今天的基线。

## 1.1. 进度概览

| 检查项（今天实测） | 结果 |
| --- | --- |
| `gofmt -l .` | ✓ 无输出，格式干净 |
| `go build ./...` | ✗ `internal/db/kv.go:28:6: missing function body` |
| `go vet ./...` | ✗ 同一处 |
| `go test ./...` | ✗ 两个包 build failed：`internal/db`（同一处）与 `scratch/kvprobe`（它 import 前者） |
| `go list ./...` | 四个包：`cmd/dbshell`、`internal/db`、`scratch/hello`、`scratch/kvprobe` |

代码这两天没动过（`kv.go` 的最后改动停在 10-11 22:05），所以 10-11 记下的两处待办原样还在：

| 待办 | 要点 |
| --- | --- |
| 补 `Del` | 要先查一次拿到 `deleted` —— `delete` 对不存在的键是安全空操作，不会告诉你删没删到 |
| 定 `Get` 未命中的语义 | 按 `kv_test.go` 第 19、22 行应当是 `return nil, false, nil`；现在是 `errors.New("the key is not existed!")` |
| 可选：`Set` 的 `updated` | 按课程语义写（`!exist \|\| !bytes.Equal(prev, val)`），现在写死 `true` |

> 📌 **核心要点** 今天最省事的路径：先补 `Del` 让包能编译，再把 `Get` 的未命中分支改成 `nil, false, nil`。做完 `go test ./...` 应当全绿，`scratch/kvprobe` 也会跟着跑起来 —— 那正好是你的第一个“实验区跑通”时刻。

## 1.2. 手上已有的工具（10-11 建的，今天直接可用）

1. `gokw` —— 关键字速查（`gokw` / `gokw <词>` / `gokw spec`），文件在 `~/.config/fish/functions/gokw.fish`；
2. `scratch/` —— 学习实验区：`go run ./scratch/hello` 现在就能跑，`go run ./scratch/kvprobe` 等 `Del` 补完；
3. `note/INDEX.md` —— 全部问答的索引。

背景细节在 `note/2026-10-11.typ`（4 节 19 页）：关键字怎么查、`scratch/` 目录怎么设计、`gokw` 怎么用。

# 2. Go 有没有 assert？防御性编程在 Go 里怎么做

## 2.1. 费曼直觉

`assert` 的直觉是“在这里写下必须成立的条件，一旦不成立就立刻炸在最近的地方”。Java、Python、C 都提供它。

Go **没有** —— 既没有 `assert` 关键字，也没有内建的 assert 函数，而且这是**刻意的**：Go 把“可能出问题的地方”分成两类 —— **正常会发生的错误**（用 `error` 值，调用方必须处理）和**绝不该发生的事**（`panic`）—— 然后拒绝提供“可以被关掉的检查”这第三类。

一句话记住：**Go 不是缺了 assert，是不想要它。**

## 2.2. 硬核机制

### 2.2.1. 官方态度（Go FAQ 原文）

出处：`https://go.dev/doc/faq` → “Why does Go not have assertions?”（本节 2026-10-13 抓取核对）：

```
Why does Go not have assertions?

Go doesn't provide assertions. They are undeniably convenient, but our experience
has been that programmers use them as a crutch to avoid thinking about proper error
handling and reporting. Proper error handling means that servers continue to operate
instead of crashing after a non-fatal error. Proper error reporting means that errors
are direct and to the point, saving the programmer from interpreting a large crash
trace. Precise errors are particularly important when the programmer seeing the
errors is not familiar with the code.

We understand that this is a point of contention. There are many things in the Go
language and libraries that differ from modern practices, simply because we feel
it's sometimes worth trying a different approach.
```

Listing 1: 注意这段的理由不是“assert 不好用”，而是“它会让人不去想错误该怎么处理”

### 2.2.2. 各语言断言的真实命运

| 语言 | 写法 | 生产环境里它还在吗 |
|---|---|---|
| Java | `assert cond : msg`（关键字，1.4+） | 默认**关闭**，要 `-ea` 才启用 |
| Python | `assert cond, msg`（语句） | `python -O` 时被**整条删除** |
| C / C++ | `assert(cond)` 宏 | 定义 `NDEBUG` 即移除（`static_assert` 是另一回事，那是编译期） |
| Rust | `assert!` / `debug_assert!` | `debug_assert!` 只在 debug 构建里存在 |
| **Go** | **没有** | **没有可关的东西** —— 也就不会出现“测试里在、线上没了” |

📌 **核心要点** “可以被关掉”是这类断言的共同软肋：开发时靠它兜底，生产环境一关，检查全没了。Go 干脆不提供，逼你在写的时候就把话说清楚：这是**正常错误**（返回 `error`）还是**不该发生**（`panic`）。

### 2.2.3. Go 的替代：三层防御

**第一层：错误即值 —— 编译期就逼你面对。**多返回值 + `if err != nil` + 尽早返回；comma-ok（`val, ok := m[k]`）；`errors.Is` / `errors.As` 判类型；`%w` 包装成错误链（`go doc errors.Is` 讲了它是怎么沿 `Unwrap` 走的）。

**第二层：panic / recover —— 只留给“不该发生”。**`go doc builtin panic` 的原文要点：panic 立刻停止当前 goroutine 的正常执行、跑完已 defer 的函数、一路向上“传染”，直到所有函数退出、程序以非零码结束。标准库的惯例是给这类函数加 `Must` 前缀：

```
$ go doc regexp.MustCompile
func MustCompile(str string) *Regexp
    MustCompile is like Compile but panics if the expression cannot be parsed.
    It simplifies safe initialization of global variables holding compiled
    regular expressions.
```

Listing 2: `Must` = “我知道这里不会错，错了就炸” —— 只适合包初始化这类一次性场景

**第三层：测试断言 + 静态检查 —— 把检查挪到 CI 去。**测试里用 `testing`（`go doc testing.T.Fatal`：等价于 Log + FailNow），或者项目已经在用的 `testify/assert`（**第三方**库，不是标准库）；能在编译/静态期抓的，绝不留给运行期：

```
// go vet 能抓、但编译器会放行的一类错误
name := "world"
fmt.Printf("%d\n", name)   // %d 配 string

$ go build -o /tmp/kwtest/vetcatch.bin ./vetcatch   # ✓ 编译器放行
$ go vet ./vetcatch
vetcatch/main.go:7:14: fmt.Printf format %d has arg name of wrong type string
```

Listing 3: 实测：类型没错、语法没错，所以编译器不管；go vet 一眼看穿 —— 这就是 Go 版的“编译期断言”

### 2.2.4. 真要一个 assert，它长这样

Go 源码树里就有一个（`/usr/lib/go/src/go/printer/testdata/parser.go:374`）：

```
func assert(cond bool, msg string) {
  if !cond {
    panic("go/parser internal error: " + msg)
  }
}
```

Listing 4: 普通的函数 + panic —— 这就是 Go 式断言的全部。注意名字里那句 “internal error”：它只用来表达“我们自己的不变量被破坏了”

## 2.3. 落到你的项目：防御性编程具体怎么做

### 2.3.1. 实测 1：`KV{}` 不 Open 就 Set，会怎样

```
type KV struct{ mem map[string][]byte }

func (kv *KV) Set(key, val []byte) { kv.mem[string(key)] = val }

func main() {
  kv := KV{}          // 忘了 Open(): mem 还是 nil
  defer func() { if r := recover(); r ≠ nil { fmt.Println("recovered:", r) } }()
  kv.Set([]byte("k1"), []byte("v1"))
}
```

Listing 5: 实测输出：recovered: assignment to entry in nil map —— 程序员的 bug，不是用户的错

两条防御路线（可以并用）：

1. **让零值可用**（Go 的标准库偏好这种，如 `sync.Mutex`、`bytes.Buffer`）：`Set` 里补一句 lazy init —— 这样 `KV{}` 直接就能用，`Open` 变成可选的预热；
2. **显式契约**：坚持“必须先 `Open`”，那就靠文档 + 测试把这个前提钉住（比如写一个“忘了 Open 会 panic”的测试，把行为固定下来）。

> ⚠️ **陷阱预警** 注意区别：**读** 一个 nil map 是合法的（返回零值），只有**写**才 panic。所以 `Get` 不初始化也不会炸，`Set` 会 —— 防御要加在真正危险的那一侧。

### 2.3.2. 实测 2：`Get` 未命中该不该报错

key 不存在是**正常情况**（用着用着就会遇到），不是编程错误。所以正确写法是 `return nil, false, nil` —— 用 `ok` 表达“没找到”，而不是 `error`，更不是 panic。这正是 FAQ 里说的 proper error handling：**错误应该是直接、切题的**，而不是让调用方去看一堆崩溃栈。

（你现在 `kv.go` 里返回的是 `errors.New("the key is not existed!")`，与 `kv_test.go` 第 19、22 行的期望冲突 —— 就是「当日起点」里那条待办。）

### 2.3.3. 实测 3：跨 goroutine 的 panic 兜不住

```
    func main() {
      defer func() { if r := recover(); r ≠ nil { fmt.Println("主 goroutine recovered:",
r) } }()

      done := make(chan struct{})
      go func() {
        defer close(done)
        panic("子 goroutine 里炸了")
      }()
      <-done
    }
```

Listing 6: 实测：进程直接崩，exit status 2 —— 主 goroutine 的 recover 完全没能兜住，连 done 都没关

结论：**recover 只对当前 goroutine 有效**，每个可能 panic 的 goroutine 都要有自己的 `defer recover`（HTTP 服务器就是这么做的：每个请求一个 goroutine，各兜各的）。用 panic 代替 assert 在并发代码里格外危险。

### 2.3.4. 决策表：这种情况该用哪个

| 情况 | 用什么 |
|---|---|
| 用户输入非法、key 不存在、文件打不开、网络超时 | `error` 返回值（正常错误，必须能被处理） |
| 调用方违反了 API 前置条件 | 返回 `error` 并在文档里写清；构造器 `NewX(...) (*X, error)` 先校验 |
| 程序内部不变量被破坏（“这不可能发生”） | `panic`（自己的 bug，就该炸在最近处） |
| 我“只想在开发时检查一下” | 写进 `go test`，别塞进生产代码 |
| 格式串、可疑写法、锁的复制、越界常量 | `go vet` / 静态检查 / `-race` |

## 2.4. 陷阱与易错

1. `_ = something()` **不是断言**：它什么都不检查，只丢弃返回值 —— 看起来像“我确认过了”，实际没有任何约束力。
2. `testify/assert` **不是标准库**：Go 标准库里没有断言包，`assert.Equal(t, ...)` 来自第三方（本项目已依赖）。它只在 `_test.go` 里用，不会混进生产代码。
3. **别在库代码里为“用户错误”panic**：panic 的语义是“这个进程/这个 goroutine 不该继续”，用来处理可预期的输入错误，会让调用方没法优雅降级。
4. `recover` **必须在** `defer` **的函数里直接调用**：`go doc builtin recover` 原文说得很清楚 —— 在 defer 调用的函数**内部**再调 recover 是无效的（那属于“不是被 defer 的函数直接调用”）。
5. **跨 goroutine 的 panic 不可 recover**（上面实测 3）：进程级事故。

6. `Must` **别滥用**：它只适合“包初始化时确定不会失败”的场景（写死的正则、写死的模板）。放在请求路径上就是把可恢复错误变成崩溃。

## 2.5. 关键点

1. Go 没有 `assert`，官方 FAQ 有一节专门解释（“programmers use them as a crutch to avoid thinking about proper error handling”），这是设计选择而非缺失。
2. 其它语言的断言大多**可关闭**（Java `-ea`、Python `-O`、C `NDEBUG`、Rust `debug_assert!`）；Go 用“error 或 panic 二选一”替代，两者都不可关闭。
3. 防御的三层落点：`error`（正常错误）→ `panic/recover`（不该发生）→ 测试与 `go vet`（把检查前移到 CI）。
4. Go 式断言就是一个普通函数 + `panic`，GOROOT 里就有现成例子（`go/printer` 的 `assert`）。
5. 你项目里最实际的两条：`Set` 要么 lazy init、要么把 `Open` 写进契约；`Get` 未命中要用 `ok` 而不是 `error`。

## 2.6. 变式检验

1. 我想让断言“只在开发时生效”？→ 用 `go test` 里的断言（测试本来就不进生产二进制），别在业务代码里搞条件编译。
2. `panic` 能让服务更安全吗？→ 不会。HTTP 服务之所以不会因为一个请求崩掉，是因为**每个请求自己 recover**，而不是因为 panic 安全。
3. `errors.New` 和 `panic` 该怎么选？→ 问自己：“调用方有没有可能合理地处理它？”能 → `error`；不能（说明是调用方违反了契约）→ 考虑 panic。
4. 为什么标准库用 `MustCompile` 而不返回 error？→ 那是包初始化期、表达式是写死的常量；如果换成用户输入的正则，就该用 `regexp.Compile` 返回 error。

## 2.7. 同类归类

四种“出错处理”手段的分工：

| 手段 | 谁写 | 用途 |
|---|---|---|
| `error` 返回值 | 被调用方返回，调用方处理 | 可预期的失败：输入、IO、网络、key 不存在 |
| `panic` / `recover` | 被调用方触发，边界处兜 | 不该发生的内部错误；包边界防止拖垮整个进程 |
| 测试断言（`testing` / testify） | 你写在 `_test.go` 里 | 验证行为，不参与生产运行 |
| `go vet` / `-race` / fuzz | 工具链 | 编译期或 CI 期发现“人类常犯的错” |

## 2.8. Go 语言机制补充

**panic 在运行时怎么实现**：调用 `panic` 会走到运行时的 `gopanic`（`/usr/lib/go/src/runtime/panic.go`），它把当前 goroutine 的 defer 链逐个执行（这也是为

什么 `defer` 一定跑得到）、带上 `_panic` 结构一路向上；goroutine 栈全部退完还没人 `recover`，运行时打印栈并让进程以非零码退出。这就是实测 3 里“exit status 2”的来源。

**`recover` 的生效条件**：只能在**被 defer 直接调用**的函数里生效 —— 这不是编译期强制，而是运行时的判定：`gopanic` 只在执行 defer 链的过程中查看是否有人调用 `recover`。

**可推导 vs 只能记**：`panic` 的行为可以从 `go doc builtin panic` 与 runtime 源码推出来；而“为什么 Go 不提供 assert”是**设计取向**，推不出来 —— 它来自官方 FAQ 里那段经验之谈。记忆锚点：**能做成 error 的，Go 不会做成断言。**

**验证命令**：

```
$ go doc builtin panic      # panic 的完整语义（含 Go 1.21 对 panic(nil) 的改动）
$ go doc builtin recover    # recover 的生效条件
$ go doc regexp.MustCompile # 标准库的 Must 惯例
$ go doc errors.Is          # 错误链怎么判
$ go vet ./...              # 编译期防御：编译器放行的、它来抓
```

Listing 7: 都能离线跑，出处就是本机的 Go 1.27.1 发行版

## 2.9. 本次操作与当前状态

### 新增的实测程序（在 `/tmp/kwtest/`，未进仓库）

1. `nilmap/` —— `KV{}` 未 `Open` 就 `Set`：`recovered: assignment to entry in nil map`；
2. `vetcatch/` —— `%d` 配 `string`：编译器放行、`go vet` 抓住；
3. `gopanic/` —— 子 goroutine panic + 主 goroutine recover：进程崩溃，`exit status 2`。

### 引用的权威出处

1. Go 官方 FAQ “Why does Go not have assertions?”（`https://go.dev/doc/faq`，本轮抓取核对，原文见上）；
2. 本机 Go 1.27.1：`go doc builtin panic` / `builtin recover` / `regexp.MustCompile` / `testing.T.Fatal` / `errors.Is`；
3. GOROOT 源码：`/usr/lib/go/src/go/printer/testdata/parser.go:374`（手写 `assert` 的实例）、`/usr/lib/go/src/runtime/panic.go`。

**没有改动**：项目代码一行没动（本次只做调研与实测）。`kv.go` 的两条待办照旧，见上面「当日起点」。

# 3. 把 defer 探针代码放进 scratch/defer/

## 3.1. 这次做了什么

你发来一张截图（截自 `note/2026-10-07.typ` 的「`defer kv.Close()` 是什么意思」→ 3.2.2 实测），上面有 4 个探针函数：`order`（LIFO 顺序）、`argTiming`（参数求值时机）、`closureTiming`（闭包）、`namedReturn`（命名返回值）。要求写进 `scratch/defer/main.go`。

截图在第 28 行被截断（标题写的是“五种行为”，第 ⑤ 种 `panic` 情形没显示完），那一段按笔记里记录的同一次实测补写：

```
func panicCase() {
  defer func() { fmt.Println("  recover 到:", recover()) }()
  defer fmt.Println("  panic 前注册的 defer 仍然执行")
  panic("出事了")
}
```

Listing 8: 顺序要点：后注册的先执行（LIFO），所以 recover 那个 defer 要先注册，才会排在「打印一行」的后面

## 3.2. 结果：输出与笔记逐字一致

```
$ go run ./scratch/defer
① LIFO 顺序:
  函数体结束
  defer 3
  defer 2
  defer 1
② 参数求值时机:
  函数体里 i = 100
  defer 捕获 i = 0
③ 闭包 vs 参数:
  闭包读 i = 100
④ 命名返回值: 11
⑤ panic 情形:
  panic 前注册的 defer 仍然执行
  recover 到: 出事了
```

Listing 9: ② 的 0 与 ③ 的 100 是 defer 最经典的差别：参数在 defer 语句执行时定格，闭包到真正执行时才读

验证三连：`go run ./scratch/defer` ✓（输出如上）、`gofmt -l scratch/` 无输出 ✓、`go vet ./scratch/defer` ✓。

> 📌 **核心要点** 这段代码现在有了固定住处。以后想改一个数字看行为变化（比如把 ③ 的闭包改成传参形式 `defer func(i int) { ... }(i)`），直接 `go run ./scratch/defer` 就行 —— 不必回 PDF 里翻。

### 3.3. 本次操作与当前状态

1. 新增 `scratch/defer/main.go`（`package main` + `import "fmt"` + `main()` + 5 个探针函数），并给 `scratch/README.md` 的「现有实验」表补了 `defer/` 一行；
2. 没有改动项目代码；`kv.go` 的两条待办（补 `Del`、`Get` 未命中改成 `nil, false, nil`）照旧，见「当日起点」。

# Index of Listings

Listing 1 注意这段的理由不是“assert 不好用”，而是“它会让人不去想错误该怎么处理” ..... 4
Listing 2 `Must` = “我知道这里不会错，错了就炸” —— 只适合包初始化这类一次性场景 ..... 5
Listing 3 实测：类型没错、语法没错，所以编译器不管；go vet 一眼看穿 —— 这就是 Go 版的“编译期断言” ..... 5
Listing 4 普通的函数 + panic —— 这就是 Go 式断言的全部。注意名字里那句 “internal error”：它只用来表达“我们自己的不变量被破坏了” ..... 5
Listing 5 实测输出：recovered: assignment to entry in nil map —— 程序员的 bug，不是用户的错 ..... 6
Listing 6 实测：进程直接崩，exit status 2 —— 主 goroutine 的 recover 完全没能兜住，连 done 都没关 ..... 7
Listing 7 都能离线跑，出处就是本机的 Go 1.27.1 发行版 ..... 9
Listing 8 顺序要点：后注册的先执行（LIFO），所以 recover 那个 defer 要先注册，才会排在「打印一行」的后面 ..... 10
Listing 9 ② 的 0 与 ③ 的 100 是 defer 最经典的差别：参数在 defer 语句执行时定格，闭包到真正执行时才读 ..... 10